UniSENAI

Trilha de Banco de Dados

Módulo 02: Estruturação de Banco de Dados

Instruções para a melhor prática de Estudo

1. **Leia atentamente todo o conteúdo:** Antes de iniciar qualquer atividade, faça uma leitura detalhada do material fornecido na trilha, compreendendo os conceitos e os exemplos apresentados.

2. **Não se limite ao material da trilha:** Utilize o material da trilha como base, mas busque outros materiais de apoio, como livros, artigos acadêmicos, vídeos, e blogs especializados. Isso enriquecerá o entendimento sobre o tema.

3. **Explore a literatura:** Consulte livros e publicações reconhecidas na área, buscando expandir seu conhecimento além do que foi apresentado. A literatura acadêmica oferece uma base sólida para a compreensão de temas complexos.

4. **Realize todas as atividades propostas:** Conclua cada uma das atividades práticas e teóricas, garantindo que você esteja aplicando o conhecimento adquirido de maneira ativa.

5. **Evite o uso de Inteligência Artificial para resolução de atividades:** Utilize suas próprias habilidades e conhecimentos para resolver os exercícios. O aprendizado vem do esforço e da prática.

6. **Participe de debates:** Discuta os conteúdos estudados com professores, colegas e profissionais da área. O debate enriquece o entendimento e permite a troca de diferentes pontos de vista.

7. **Pratique regularmente:** Não deixe as atividades para a última hora. Pratique diariamente e revise o conteúdo com frequência para consolidar o aprendizado.

8. **Peça feedback:** Solicite o retorno dos professores sobre suas atividades e participe de discussões sobre os erros e acertos, utilizando o feedback para aprimorar suas habilidades.

Essas instruções são fundamentais para garantir um aprendizado profundo e eficaz ao longo das trilhas.

Módulo 2: Estruturação de Bancos de Dados

1. Modelo de Entidade e Relacionamento (MER)

O **Modelo de Entidade e Relacionamento (MER)** é uma etapa fundamental no projeto de bancos de dados relacionais. Ele permite **planejar, visualizar e organizar** as informações que serão armazenadas, bem como os **relacionamentos entre os dados**, antes da implementação no SGBD.

O uso do MER contribui para:

- Melhor entendimento do domínio do sistema
- Redução de erros de modelagem
- Maior integridade e consistência dos dados
- Facilidade de manutenção e evolução do banco

1.1 Entidades

Entidades representam objetos ou conceitos do mundo real que possuem existência própria dentro do sistema.

- **Exemplo Prático**:
  - Entidade: `Cliente`
    - **Atributos**:
      - `id_cliente` (Chave Primária)
      - `nome` (VARCHAR)
      - `email` (VARCHAR)
      - `telefone` (VARCHAR)
  - Entidade: `Produto`
    - **Atributos**:
      - `id_produto` (Chave Primária)
      - `nome` (VARCHAR)
      - `preco` (DECIMAL)
      - `estoque` (INT)

Boas Práticas

- Sempre defina uma **chave primária única** para cada entidade.
- Utilize nomes claros e objetivos.
- Evite atributos redundantes, como `preco_produto` e `valor_produto` quando ambos representam a mesma informação.

1.2 Atributos

Os **atributos** representam as características de uma entidade e descrevem os dados que serão armazenados.

1. **Exemplo Prático**:
    a. Atributos de `Pedido`:
        i. `id_pedido` (Chave Primária)
        ii. `data_pedido` (DATE)
        iii. `id_cliente` (Chave Estrangeira)

Boas Práticas

- Utilize nomes que reflitam claramente a natureza do dado.
- Escolha tipos de dados adequados para otimizar armazenamento e desempenho.
- Exemplo: prefira `VARCHAR(100)` para nomes em vez de `TEXT`, quando possível.

1.3 Relacionamentos

Relacionamentos descrevem como as entidades se conectam entre si dentro do sistema.

Exemplos Práticos

- **Cliente realiza Pedido**
  Tipo: **1:N** (um cliente pode realizar vários pedidos)
- **Produto pertence a Categoria**
  Tipo: **N:1** (vários produtos pertencem a uma categoria)

Boas Práticas

- Nomeie os relacionamentos de forma clara e sem ambiguidade.
- Defina se o relacionamento é **obrigatório ou opcional** (exemplo: nem todo cliente precisa ter pedidos cadastrados).

1.4 Cardinalidades

A **cardinalidade** define quantas ocorrências de uma entidade podem se relacionar com outra.

Tipos Comuns

- **1:1** - Um Cliente possui um Endereço Fiscal.
- **1:N** - Um Cliente pode realizar vários Pedidos.
- **N:M** - Um Produto pode estar em vários Pedidos e um Pedido pode conter vários Produtos.

Boas Práticas

- As cardinalidades devem refletir fielmente as regras do negócio.
- Relacionamentos **N:M** devem sempre ser resolvidos com uma **tabela intermediária**
  Exemplo: `pedido_produto`.

1.5 Modelo Lógico e Físico

Modelo Lógico

- Representa as entidades, atributos e relacionamentos.
- Independe do SGBD.
- Geralmente ilustrado por diagramas (DER).
- Pode ser criado no MySQL Workbench.

Modelo Físico

- Representa a implementação real no SGBD.
- Define:
  - Tipos de dados
  - Chaves primárias e estrangeiras
  - Restrições (constraints)
- É implementado por meio de comandos SQL.

```sql
CREATE TABLE clientes (
    id_cliente INT AUTO_INCREMENT PRIMARY KEY,
    nome VARCHAR(100),
    email VARCHAR(100),
    telefone VARCHAR(15)
);
```

---

2. Criação e Gerenciamento de Bancos de Dados

Para gerenciar um banco de dados é essencial dominar comandos básicos que permitem criar, excluir e selecionar um banco de dados. Vamos entender cada um deles.

2.1 Comando CREATE DATABASE

O comando `CREATE DATABASE` é utilizado para criar um novo banco de dados.

Sintaxe:

```
CREATE DATABASE nome_do_banco;
```

Exemplo prático:

```
CREATE DATABASE loja_virtual;
```

Nesse exemplo, estamos criando um banco de dados chamado `loja_virtual`, que poderá armazenar tabelas relacionadas a uma loja online.

Comando com definição de caracteres:

```
CREATE DATABASE loja_virtual CHARACTER SET utf8mb4 COLLATE utf8mb4_general_ci;
```

Neste caso, especificamos o conjunto de caracteres e a collation (regras de classificação) para o banco de dados.

Boas Práticas:

- Dê nomes significativos ao banco de dados para facilitar a identificação.
- Utilize `CHARACTER SET` e `COLLATE` para evitar problemas de acentuação e caracteres especiais.

2.2 Comando DROP DATABASE

O comando `DROP DATABASE` remove permanentemente um banco de dados e todo o seu conteúdo.

Sintaxe:

```
DROP DATABASE nome_do_banco;
```

Exemplo prático:

```
DROP DATABASE loja_virtual;
```

Este comando excluirá permanentemente o banco de dados `loja_virtual` e todos os seus dados.

Boas Práticas:

- Antes de usar `DROP DATABASE`, certifique-se de ter um backup dos dados.

- Evite executar este comando em ambientes de produção sem uma validação rigorosa.

2.3 Comando USE

Seleciona o banco de dados que será utilizado.

Sintaxe:

```
USE nome_do_banco;
```

Exemplo prático:

```
USE loja_virtual;
```

Neste exemplo, estamos selecionando o banco de dados `loja_virtual`. A partir desse momento, qualquer comando SQL será executado dentro desse banco.

Boas Práticas:

- Verifique se o banco foi selecionado corretamente antes de executar comandos.
- Use o comando `SHOW DATABASES;` para visualizar todos os bancos de dados disponíveis.

Resumos dos Comandos

Comando	Função	Sintaxe
CREATE DATABASE	Cria um novo banco de dados	*CREATE DATABASE nome_do_banco;*
DROP DATABASE	Exclui um banco de dados	*DROP DATABASE nome_do_banco;*
USE	Seleciona um banco para uso	*USE nome_do_banco;*

Exercício Prático

1. Crie um banco de dados chamado `biblioteca`.
2. Selecione o banco `biblioteca`.
3. Exclua o banco após confirmar que não há dados importantes.

Comandos Esperados:

```
CREATE DATABASE biblioteca;
USE biblioteca;
DROP DATABASE biblioteca;
```

3. Criação e Gerenciamento de Tabelas

3.1 Comando CREATE TABLE

O comando `CREATE TABLE` cria tabelas dentro de um banco de dados.

Sintaxe:

```
CREATE TABLE nome_da_tabela (
    nome_coluna1 TIPO_DADO(RESTRIÇÃO),
    nome_coluna2 TIPO_DADO(RESTRIÇÃO),
    ...
);
```

Exemplo prático:

```
CREATE TABLE clientes (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nome VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE,
    telefone VARCHAR(15),
    data_cadastro DATE DEFAULT CURRENT_DATE
);
```

Neste exemplo, criamos uma tabela `clientes` com as seguintes colunas: `id`, `nome`, `email`, `telefone` e `data_cadastro`.

3.2 Definição de Colunas, Tipos de Dados e Restrições

Ao criar tabelas, é necessário definir as colunas, seus tipos de dados e as restrições que garantem a integridade dos dados.

3.3 Tipos de Dados

Tipo de Dado	Descrição
INT	Números inteiros
VARCHAR(n)	Texto de até n caracteres
DATE	Datas no formato AAAA-MM-DD
DECIMAL(x, y)	Números decimais com x digitos e y decimais
BOOLEAN	Valores booleanos (TRUE ou FALSE)

3.4 Restrições

Restrição	Descrição
PRIMARY KEY	Identifica unicamente cada registro
FOREIGN KEY	Cria relação com outra tabela
UNIQUE	Garante que o valor é único
NOT NULL	Impede valores nulos na coluna
DEFAULT	Define um valor padrão para a coluna

Exemplo prático:

```
CREATE TABLE pedidos (
    id INT AUTO_INCREMENT PRIMARY KEY,
    cliente_id INT NOT NULL,
    data_pedido DATE DEFAULT CURRENT_DATE,
    valor_total DECIMAL(10, 2) NOT NULL,
    FOREIGN KEY (cliente_id) REFERENCES clientes(id)
);
```

Nesse exemplo, a tabela `pedidos` faz referência à tabela `clientes` através de uma chave estrangeira (`FOREIGN KEY`).

---

4. Alteração de Tabelas (`ALTER TABLE`)

O comando `ALTER TABLE` permite modificar uma tabela existente, adicionando, removendo ou alterando colunas, bem como mudando restrições.

4.1 Adição de Colunas Sintaxe:

```
ALTER TABLE nome_da_tabela ADD nome_da_coluna TIPO_DADO(RESTRIÇÃO);
```

Exemplo prático:

```
ALTER TABLE clientes ADD endereco VARCHAR(255);
```

4.2 Remoção de Colunas Sintaxe:

```
ALTER TABLE nome_da_tabela DROP COLUMN nome_da_coluna;
```

Exemplo prático:

```
ALTER TABLE clientes DROP COLUMN telefone;
```

4.3 Alteração de Colunas Sintaxe:

```
ALTER TABLE nome_da_tabela MODIFY nome_da_coluna TIPO_DADO(RESTRIÇÃO);
```

Exemplo prático:

```
ALTER TABLE clientes MODIFY nome VARCHAR(150) NOT NULL;
```

4.4 Alterar Restrições (PRIMARY KEY, FOREIGN KEY) Para adicionar ou remover chaves primárias e estrangeiras, utilizamos os seguintes comandos:

Adicionar chave estrangeira:

```
ALTER TABLE pedidos ADD FOREIGN KEY (cliente_id) REFERENCES clientes(id);
```

Remover chave estrangeira:

```
ALTER TABLE pedidos DROP FOREIGN KEY fk_cliente_id;
```

---

5. Exclusão de Tabelas (`DROP TABLE`)

O comando `DROP TABLE` exclui uma tabela e todo o seu conteúdo permanentemente. Cuidado! Esse comando é irreversível.

Sintaxe:

```
DROP TABLE nome_da_tabela;
```

Exemplo prático:

```
DROP TABLE pedidos;
```

Esse comando exclui a tabela `pedidos` e todos os seus dados.

Exercícios práticos

1. Crie uma tabela `produtos` com as colunas `id`, `nome`, `preco`, `quantidade_estoque` e `data_cadastro`. Use as restrições adequadas (chave primária, not null, etc.).
2. Adicione uma coluna `descricao` à tabela `produtos`.
3. Altere o tipo de dados da coluna `preco` para `DECIMAL(8, 2)`.
4. Exclua a coluna `quantidade_estoque` da tabela `produtos`.
5. Exclua a tabela `produtos`.

Atividade Prática: Pesquisa sobre Normalização

Objetivo: Compreender o conceito de normalização e sua aplicação no projeto de bancos de dados.

1. **Tarefa do Acadêmico**:
   - Pesquisar sobre o processo de normalização, incluindo:
     - Conceitos das formas normais (1ª, 2ª, 3ª e BCNF).
     - Vantagens da normalização na eliminação de redundâncias e na garantia de integridade.
     - Exemplos práticos de como normalizar um banco de dados.
2. **Entrega**:
   - Elaborar uma apresentação explicando:
     - O que é normalização.
     - As formas normais e seus requisitos.
     - Um exemplo prático mostrando como transformar uma tabela não normalizada em 3ª forma normal.
3. **Formato da Apresentação**:
   - **Slides** (PowerPoint ou Google Slides) com:
     - Títulos claros.
     - Uso de gráficos ou diagramas para ilustrar o processo.
   - Duração sugerida: 10 minutos.
4. **Critérios de Avaliação**:
   - Clareza na explicação dos conceitos.
   - Qualidade visual e organização dos slides.
   - Aplicação correta do processo de normalização no exemplo.